Лабораторна Робота №4
Тема: Детекція об’єктів

Об’єднує: Тема 7

- Використання YOLOv8 та SSD для знаходження об’єктів на зображеннях і відео;
- Побудова обмежувальних рамок (bounding boxes) і підрахунок об’єктів.

Результат: вміння реалізувати object detection у практичних задачах.

У сучасних системах комп’ютерного зору детекція об’єктів є однією з ключових задач, на основі якої будуються системи відеоспостереження, автономного транспорту, робототехніки, смарт-аналізу відеопотоків, контролю виробництва, медичної діагностики та сотні інших практичних застосувань. У цій лабораторній роботі ми ознайомимося з двома популярними підходами для детекції об’єктів — моделями YOLOv8 та SSD. Ви навчитеся виконувати інференс на статичних зображеннях та відеопотоці у реальному часі, налаштовувати параметри впевненості й фільтрації (NMS), аналізувати отримані результати та порівнювати швидкість і точність різних моделей. Лабораторна робота дозволить зрозуміти принципи роботи сучасних детекторів об’єктів та застосувати їх у практичних сценаріях.

1. Використання YOLOv8 та SSD для знаходження об’єктів на зображеннях і відео

YOLO (You Only Look Once) — це сімейство сучасних моделей глибинного навчання для виявлення об’єктів на зображеннях та відео в реальному часі. Головна ідея YOLO полягає у виконанні детекції «за один прохід» через мережу, що робить її значно швидшою у порівнянні з класичними підходами, які спочатку генерували області інтересу, а потім класифікували їх окремо. Завдяки цьому YOLO демонструє високу продуктивність і здатність працювати на edge/mobile пристроях, що є критично важливим для реальних інженерних застосувань: відеоспостереження, автономні системи, промислові лінії контролю, робототехніка та AR/VR.

1.1. Використання YOLOv8 (Ultralytics)

Для початку роботи із YOLO на комп’ютері, потрібно встановити відповідний модуль Python – ultralytics. Виконайте наступну команду у консолі для встановлення модуля:

```
pip install ultralytics
```

1.1.1. Швидкий старт: інференс на зображенні

Першим прикладом, який ми розглянемо із YOLO буде інференс на зображенні. Інференс на зображенні — це етап, на якому попередньо навчену модель застосовують до нових даних, щоб отримати передбачення без повторного навчання. Тобто, модель вже має сформовані знання (ознаки, патерни) з етапу тренування, і тепер вона аналізує нове зображення, знаходить на ньому об’єкти та повертає результати: координати обмежувальних рамок, назви класів та рівень впевненості. На практиці це перший крок

реального використання моделі детекції — ми беремо будь-яке нове зображення, завантажуємо його в мережу та перевіряємо, наскільки коректно модель може розпізнавати об’єкти поза тренувальним набором даних.

Для початку роботи із першим прикладом створіть новий файл, який буде містити код для виконання інференсу на зображенні.

Далі у новому файлі почніть введення коду із імпортування бібліотек необхідних для виконання даного прикладу:

```
from ultralytics import YOLO
import cv2
```

Наступним кроком є ініціалізація моделі YOLO для завантаження. Ми будемо використовувати версію 11 версію YOLO останню та актуальну (на момент цієї лабораторної). YOLO11 базується на універсальному модельному ряді, представленому в YOLOv8, пропонуючи розширену підтримку для різних завдань комп'ютерного зору. Дана модель має список варіантів:

- yolo11n.pt;
- yolo11s.pt;
- yolo11m.pt;
- yolo11l.pt;
- yolo11x.pt;

Для даного прикладу виберіть yolo11n.pt:

```
model = YOLO("yolo11n.pt")
```

Далі налаштовуємо інференс та вказуємо зображення для аналізу. Виберіть будь-яке зображення (наприклад, зображення із першої лабораторної роботи), яке б хотіли проаналізувати. Встановлюємо поріг впевненості рівний 0.25 та IoU рівний 0.45:

```
img_path = "cat.jpg"
results = model(img_path, conf=0.25, iou=0.45)
```

Поріг впевненості – це мінімальна впевненість моделі, з якою вона вважає, що знайдений об’єкт є справжнім. Якщо модель думає з імовірністю нижче 0.25 — цей бокс не показується. Показник IoU (Intersection over Union) показує наскільки сильно перекриваються два бокси. Тут iou=0.45 означає, що якщо два бокси перекриваються більше ніж на 45%, то з двох схожих результатів YOLO залишить лише той, що має більшу впевненість.

І останнім кроком – візуалізуємо зображення із обмежувальними рамками та показником детекції:

```
for r in results:
    im = r.plot()  # намальовані бокси/класи/ймовірності
    cv2.imshow("YOLOv11", im)
    cv2.imwrite("yolo_out.jpg", im)
    cv2.waitKey(0)
cv2.destroyAllWindows()
```

Збережіть файл та виконайте його. У результаті виконання даного коду, з’явиться вікно із тим самим зображенням, та рамками навколо об’єкта із показником впевненості (Рис. 1.1).

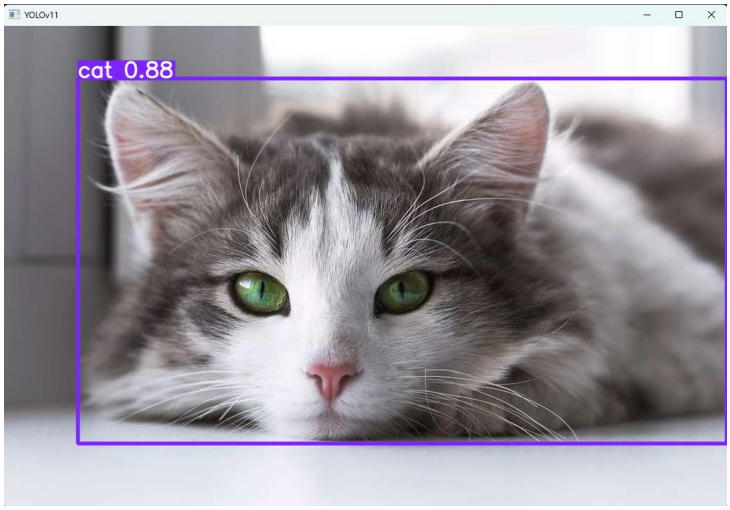

Рис. 1.1. Отримане зображення із детекцією об’єктів

Проаналізуйте отриманий результат. Чи був об’єкт класифікований правильно, та з яким показником впевненості? Спробуйте виконати даний код використовуючи інший варіант моделі YOLO11 зі списку. Також спробуйте виконати даний код із різним показником порогу впевненості та показником перекриття. Порівняйте результати між собою, та визначте, який саме варіант моделі показав для вас найкращий показник впевненості.

1.1.2. Відео/вебкам інференс

Наступним прикладом, який ми розглянемо із YOLO буде детекція об’єктів на відео чи в прямому записі з камери. Відео/вебкам інференс дозволяє виконувати детекцію об’єктів у реальному часі, аналізуючи кожен кадр відеопотоку так само, як звичайне одиночне зображення. На відміну від статичного прикладу, де модель працює з однією фотографією, робота з відео вимагає безперервної обробки послідовності кадрів і оптимізованої продуктивності, оскільки швидкість виводу результату безпосередньо впливає на користувацький досвід. Це є важливим кроком до практичного застосування детекції: системи спостереження, робототехніка, контролю виробництва, автономні платформи та взаємодія людина-комп’ютер. У даній підтемі ви навчитеся підключати модель до відеопотоку(або відео, якщо відеопотік не є можливим) та реалізовувати аналіз об’єктів у динаміці.

Для початку роботи із першим прикладом створіть новий файл, який буде містити код для виконання інференсу на прикладі відео.

Далі у новому файлі почніть введення коду із імпортування бібліотек необхідних для виконання даного прикладу:

```
from ultralytics import YOLO
import cv2, time
```

В даному прикладі також імпортуємо модуль time для підрахунку кадрів на секунду в реальному часі. Далі ініціалізовуємо модель YOLO, версії 11 як і в попередньому випадку та стартуємо таймер для підрахунку кадрів відео:

```
model = YOLO("yolo11n.pt")
cap = cv2.VideoCapture("planes.mp4")  # 0 - вебкам; або "video.mp4"
prev = time.time()
```

У нашому випадку ми будемо аналізувати відео planes.mp4, яке містить відео літаків у небі, які модель старається розпізнати. Якщо змінити вхідний параметр *cv2.VideoCapture* на 0, ми отримаємо можливість використовувати камеру, як джерело відео, та аналізувати потокове відео.

Наступний код запускає процес аналізу відео у циклі, доки не досягне кінця відео – і тоді цикл зупиниться та програма завершиться:

```
while True:
    ok, frame = cap.read()
    if not ok: break
    results = model(frame, conf=0.3, iou=0.5, verbose=False)
    out = results[0].plot()

    now = time.time()
    fps = 1.0 / (now - prev)
    prev = now
    cv2.putText(out, f"FPS: {fps:.1f}", (10,30),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)

    cv2.imshow("YOLOv11 live", out)
    if cv2.waitKey(1) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

У цьому коді ми створюємо вікно для відображення відео *cv2.imshow("YOLOv11 live", out)*, та виводимо додатково текст із числом кадрів на секунду *cv2.putText(out, f"FPS: {fps:.1f}", (10,30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)*.

Збережіть файл та виконайте його. У результаті виконання даного коду, з'явиться вікно із тим із відео та рамками навколо об'єктів із показником впевненості, які модель намагається розпізнати (Рис. 1.2).

Рис. 1.2. Скріншот із відео, в якому модель намагається розпізнати об'єкти

Проаналізуйте отриманий результат. Чи був об'єкт класифікований правильно, та з яким показником впевненості? Зауважте наскільки добре модель може розпізнати об'єкти коли вони перетинаються. Спробуйте виконати даний код використовуючи інший варіант моделі YOLO11 зі списку. Також спробуйте виконати даний код із різним показником порогу впевненості та показником перекриття. Порівняйте результати між

собою, та визначте, який саме варіант моделі показав для вас найкращий показник впевненості, та чи була різниця в ступені перекриття чи класифікації при різних показниках впевненості та перекриття.

1.1.3. Тренування на власному датасеті

Далі ми розглянемо тренування YOLO моделі, та як тренування може змінити результат розпізнавання зображень. Спершу, ми виконаємо розпізнавання набору зображень, щоб зафіксувати результати розпізнавання, потім проведемо навчання моделі, і фінальне розпізнавання набору зображень, щоб перевірити чи змінилась якість розпізнавання.

Для прикладу розглянемо наступне зображення (Рис. 1.3).

Рис. 1.3. Тестове зображення для прикладу із навчанням моделі

Запустивши розпізнавання попередньо для цього прикладу було отримано наступний результат (Рис. 1.4).

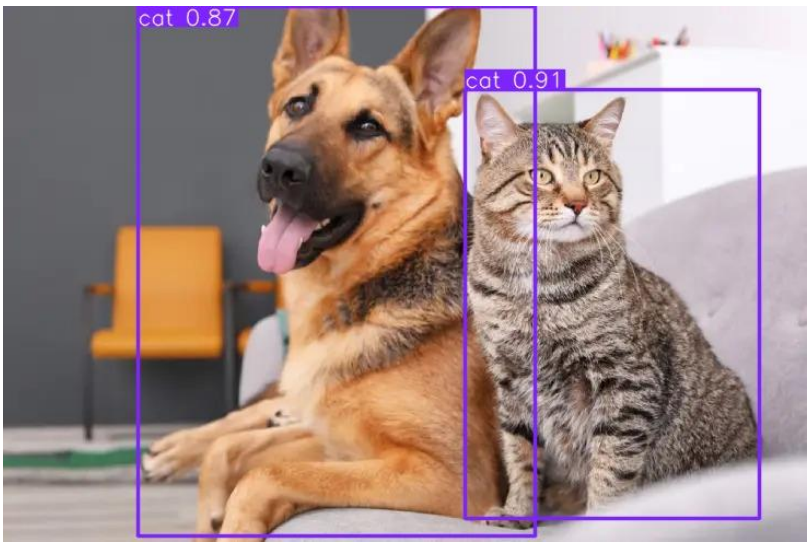

Рис. 1.4. Тестове зображення для прикладу із навчанням моделі

Як можна помітити на зображенні, модель розпізнає собаку як кота із показником впевненості 0.87, що є достатньо високим показником (діапазон 0-1). Тому в межах цього прикладу спробуємо покращити розпізнавання та допомогти моделі правильно розпізнати зображення.

Для початку роботи створіть новий файл, який буде містити код для виконання даного прикладу. Далі потрібно вказати код для завантаження залежностей та ініціалізації моделі:

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")
```

Наступним кроком проведемо перше розпізнавання для визначення наскільки добре модель може розпізнати об'єкти без додаткового налаштування:

```
model.predict(source="test_images/", conf=0.25, iou=0.5, save=True,
name="Run1")
```

У папці test_images містяться зображення для тестування. В даному випадку у папці міститься одне зображення собаки і кота (Рис. 1.3). В результаті розпізнавання було отримано зображення, в якому собака була розпізнана як кота (Рис. 1.4), про що вже говорили попередньо.

Далі проводимо тренування моделі на навчальному наборі даних:

```
model.train(
    data="german_shepherd/data.yaml",
    epochs=50,
    imgsz=640,
    batch=16,
    lr0=0.001,
    device="cpu",  # або "cpu" | "0"
)

model.val()
```

Цей набір містить тільки зображення собаки, а саме німецької вівчарки, для того, щоб модель могла краще розпізнати вівчарку на нашому зображенні. Для даного прикладу ми використовуємо 50 епох, що має бути достатньо для навчання моделі. Потрібно зазначити, що в результаті тренування тільки на одному наборі даних з одним об'єктом модель переопридiляє всю голова-класифікатор на 1 клас - *german_shepherd*, а всі старі класи (в тому числі *cat*) викидається повністю.

Після процесу тренування, в нас створюється папка runs/detect, яка містить результати тренування, та нові, перевизначені ваги, які ми отримали для класу *german_shepherd*. Використовувати модель ми далі не можемо, оскільки вона не містить оновлених ваг, і буде давати той самий результат, що і спочатку. Тому нам потрібно підвантажити нові ваги:

```
ft = YOLO("runs/detect/train/weights/best.pt")
```

І отримавши новий екземпляр моделі з оновленими вагами, ми можемо запустити новий тест:

```
ft.predict(source="test_images/", conf=0.1, iou=0.1, save=True,
name="Run3")
```

Зауважте, що цього разу ми використовуємо значно менші показники впевненості та перетину рамок об'єктів – 0.1. Це зроблено для того, щоб показати повний результат такого навчання та вивести всі об'єкти розпізнавання. В результаті нового тестування, буде отримано наступний результат (Рис. 1.5):

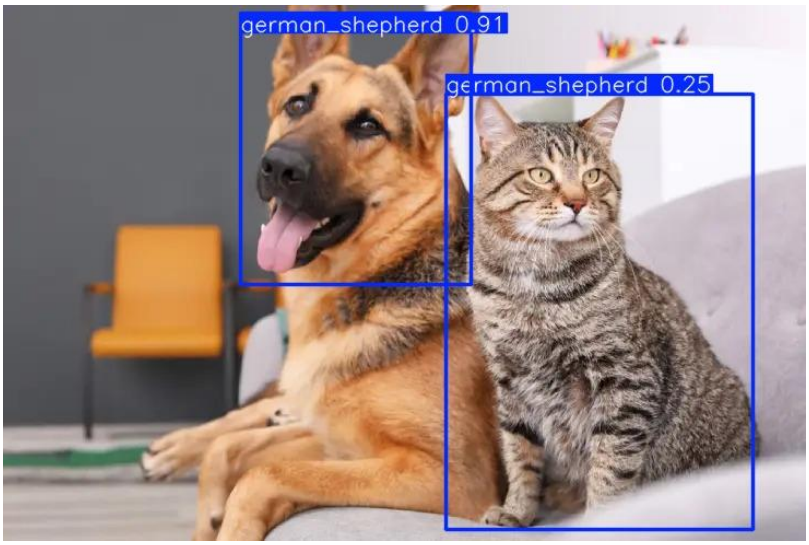

Рис. 1.5. Оновлене розпізнавання об’єктів після тренування

Цього разу, собака вже точніше розпізнається із впевненістю 0.91, та маємо навіть оновлений клас *german_shepherd* (оскільки ми тренували модель на такому класі). Проте, ми також отримали і сайд-ефект у вигляді того, що тепер кіт розпізнається із впевненістю 0.25 як *german_shepherd*. Як вже вказувалося, даний результат був отриманий оскільки ми перевизначили голову-класифікатор, та видалили попередні класи. Для того, що залишити можливість розпізнавати кота, потрібно навчати модель на такому наборі, який містив би зображення і кота і собаки.

Проаналізуйте результат, який ви отримали. Спробуйте навчати модель на іншій кількості епох, та показника впевненості. Порівняйте результати між собою, та визначте, який саме варіант моделі показав для вас найкращий показник впевненості для собаки та кота.

1.1.4. Оцінка й помилки

Після етапу тренування модель потрібно не лише протестувати на val, а й детально проаналізувати якість роботи, визначити типові помилки та зрозуміти, де саме модель працює недостатньо добре. У практичних задачах це один із найважливіших етапів після навчання, тому що саме він показує, що саме потрібно покращувати: дані, аугментації, баланс класів, архітектуру або гіперпараметри.

Тому наступним прикладом ми розглянемо візуалізацію та аналіз оцінок і помилок. Для цього нам потрібно завантажити ваги із попереднього тренування та проаналізувати їх використовуючи зображення для валідації.

Для початку роботи із першим прикладом створіть новий файл, який буде містити код для аналізу оцінок та помилок на прикладі зображень для валідації із датасету.

Далі у новому файлі почніть введення коду із імпортування бібліотек необхідних для виконання даного прикладу:

```
from ultralytics import YOLO
import cv2, os
```

Для отримання оцінок показників додайте наступний код:

```
metrics = model.val()
print("mAP@[.5:.95] =", metrics.box.map)
```

```
print("mAP@0.50     =", metrics.box.map50)
print("Precision    =", metrics.box.mp)
print("Recall       =", metrics.box.mr)
```

Для збору прикладів хибних спрацьовувань та пропусків потрібно використати зображення для валідації із набору зображень, який ми використовували для тренування моделі:

```
val_images = " german_shepherd/valid/images"
os.makedirs("errors/FP", exist_ok=True)
os.makedirs("errors/FN", exist_ok=True)

for img_name in os.listdir(val_images):
    img_path = os.path.join(val_images, img_name)
    res = model.predict(img_path, conf=0.25, iou=0.5, verbose=False)[0]

    for b in res.boxes:
        conf = float(b.conf.cpu().numpy())
        if conf > 0.80:
            cv2.imwrite(os.path.join("errors/FP", img_name),
res.plot())
        if conf < 0.30:
            cv2.imwrite(os.path.join("errors/FN", img_name),
res.plot())
```

Проаналізуйте отриманий результат. Які зображення та чому отримали хибні спрацювання чи були пропущені? Які зображення отримали найбільшу оцінку? Чи є якісь результати оцінювання, які найбільше виділяються та чому?

1.1.5. Експорт оновленої моделі

Після того як модель була навчена та протестована, наступним кроком є її підготовка до використання поза середовищем навчання — у реальних застосуваннях. Експорт і деплой означає перетворення моделі у формат, який може бути виконаний на різних платформах: мобільних пристроях, вбудованих системах, серверних сервісах, edge-обладнанні. Для цього модель можна конвертувати у стандартизовані формати, такі як ONNX, TFLite або NCNN, які забезпечують менший розмір, кращу оптимізацію швидкості, і можливість запускати інференс без повного фреймворку PyTorch.

Також розглянемо експорт YOLO-моделі в інші формати, тестування продуктивності після експорту, аналіз вплив квантизації на точність і швидкість, а також важливість оптимізації моделей для реального продакшн використання. Це є важливим завершальним етапом у життєвому циклі моделі — від тренування до реального впровадження.

Для початку роботи із першим прикладом створіть новий файл, який буде містити наш код.

Далі у новому файлі почніть введення коду із імпортування бібліотек необхідних для виконання даного прикладу:

```
from ultralytics import YOLO
```

Спочатку необхідно завантажити ваги оновленої моделі після навчання, яку ми хочемо зберегти:

```
model = YOLO("runs/detect/train/weights/best.pt")
```

Далі додаємо код для формату, в якому хочемо зберегти модель. Наприклад, для збереження у ONNX, який маю динамічні розміри увімкнутими для зручності деплою:

```
model.export(format="onnx", opset=12, dynamic=True)
```

Цілком можливо зберегти модель з оновленими вагами у форматі TFLite, але дуже часто це призводить до помилок, через проблеми сумісності різних форматів, які використовуються для конвертації у формат TFLite. Тому формат TFLite в межах цього прикладу лабораторної роботи розглядатися не буде.

NCNN формат для мобільних/вбудованих з ncnn:

```
model.export(format="ncnn")
```

В результаті виконання файлу, ви отримаєте файл у відповідному форматі. Порівняйте час, необхідний для експорту моделі та розмір створених моделей.

1.2. Використання SSD

SSD (Single Shot MultiBox Detector) — це одна з перших моделей, яка забезпечила реальний баланс між точністю та високою швидкістю детекції об'єктів. На відміну від підходів із регіональними пропозиціями, SSD виконує виявлення за один прохід мережі, оцінюючи об'єкти на кількох масштабах ознак одночасно. У цій частині лабораторної роботи ми розглянемо, як використовувати SSD разом з OpenCV DNN, що дозволяє запускати модель без PyTorch або TensorFlow. Такий підхід є дуже зручним для простих додатків, легких embedded-проєктів, використання на CPU та для задач, де важлива швидкість виконання і низькі залежності.

1.1.1. Інференс на зображенні

Першим прикладом, який ми розглянемо із SSD буде інференс на зображенні. В цілому, даний приклад аналогічний прикладу із YOLO, тільки тепер ми будемо використовувати OpenCV, локальну модель та інше зображення для тесту.

Для початку роботи із першим прикладом створіть новий файл, який буде містити наш код, та імпортуємо необхідні залежності:

```
import cv2
```

Наступним кроком буде додати модель та її опис (будемо використовувати MobileNet), які ми будемо використовувати для інференсу:

```
model_path = "ssd_mobilenet_v3.pb"
config_path = "ssd_mobilenet_v3.pbtxt"
```

Далі використовуючи файл із класами об'єктів створюємо список у пам'яті:

```
classLabels = []
filename = 'labels.txt'
with open(filename, 'rt') as spt:
    classLabels = spt.read().rstrip('\n').split('\n')
```

Далі ініціалізовуємо модель:

```
net = cv2.dnn_DetectionModel(model_path, config_path)
net.setInputSize(320, 320)
net.setInputScale(1.0/127.5)
net.setInputMean((127.5, 127.5, 127.5))
net.setInputSwapRB(True)
```

Вказуємо, яке зображення будемо використовувати для класифікації об'єктів:

```
img = cv2.imread("ssd_test_images/test_image.png")
```

Запускаємо модель для розпізнавання об'єктів:

```
        classes, scores, boxes = net.detect(img, confThreshold=0.5,
nmsThreshold=0.45)
```

Аналізуємо результат розпізнавання, додаємо рамки та підписи до розпізнаних об’єктів, зберігаємо все у файлі та виводимо фінальне зображення із розпізнаним об’єктом:

```
        if len(classes) != 0:
            for cls, score, box in zip(classes.flatten(), scores.flatten(),
boxes):
                x,y,w,h = box
                cv2.rectangle(img, (x,y), (x+w,y+h), (0,255,0), 2)
                label = f"{classLabels[cls-1]} {score:.2f}"
                cv2.putText(img, label, (x, y-6),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,0,255), 2)

        cv2.imshow("SSD", img)
        cv2.imwrite("ssd_out.jpg", img)
        cv2.waitKey(0)
        cv2.destroyAllWindows()
```

Проаналізуйте отриманий результат. Чи були правильно розпізнано об’єкти на зображенні та з якою впевненістю? Спробуйте також використати зображення із попередніх прикладів цієї лабораторної та проаналізуйте результати.

1.2.2. Відео/вебкам інференс

Наступним прикладом, який ми розглянемо із SSD буде детекція об’єктів на відео. В цілому суть прикладу повністю аналогічний прикладу із YOLO, тільки використовувати будемо OpenCV та модель MobileNet.

Для початку роботи із першим прикладом створіть новий файл, який буде містити код для виконання інференсу на прикладі відео.

Далі у новому файлі почніть введення коду із імпортування необхідних бібліотек, додавання моделі, завантаження списку класів та ініціалізації моделі, що є аналогічним попередньому прикладу.

Далі як і в попередньому прикладі та для розпізнавання з YOLO, ми вказуємо відео для аналізу, запускаємо передбачення відео, та формуємо вивід рамок, підписів та кількості кадрів на секунду у фінальному відео:

```
        cap = cv2.VideoCapture("football.mp4")  # 0 - вебкам; або "planes.mp4"
| "test_video.mp4" | "football.mp4"
        prev = time.time()

        while True:
            ok, frame = cap.read()
            if not ok: break
            classes, scores, boxes = net.detect(frame, confThreshold=0.5,
nmsThreshold=0.45)

            if len(classes) != 0:
                for cls, score, box in zip(classes.flatten(), scores.flatten(),
boxes):
                    x,y,w,h = box
```

```
                    cv2.rectangle(frame, (x,y), (x+w,y+h), (0,255,0), 2)
                    cv2.putText(frame, f"{classLabels[cls-1]} {score:.2f}",
                                (x, y-6), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
(0,0,255), 2)

            now = time.time(); fps = 1.0 / (now - prev); prev = now
            cv2.putText(frame, f"FPS: {fps:.1f}", (10,30),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (255,255,0), 2)

            cv2.imshow("SSD live", frame)
            if cv2.waitKey(1) & 0xFF == 27:
                break

        cap.release()
        cv2.destroyAllWindows()
```

Проаналізуйте отриманий результат. Чи були правильно розпізнано об’єкти на зображенні та з якою впевненістю? Спробуйте також використати інші запропоновані відео та проаналізуйте результати.

2. Побудова обмежувальних рамок (bounding boxes) і підрахунок об’єктів

2.1. Ручне створення та візуалізація боксів

Під час роботи з моделями детекції ви повинні розуміти, що обмежувальні рамки — це не просто результат моделі, а низькорівнева геометрична структура, яку можна обробляти та візуалізувати вручну. Обмежувальні рамки у форматі [x, y, w, h] описує положення об’єкта на зображенні: координати верхнього лівого кута (x, y) та ширина/висота (w, h). Це базовий універсальний формат для опису об’єктів у задачах комп’ютерного зору, який використовують як у наборах даних (датасетах), так і у післяобробці результатів моделей.

У цій підтемі ви навчитеся самостійно будувати та малювати такі бокси на зображенні, додавати до них підпис класу та значення довіри (confidence), а також змінювати колір візуалізації. Це важливо для розуміння постобробки: саме цей етап виконується після інференсу моделі, і він відповідає за перетворення числових координат у наочну, зрозумілу людині візуалізацію результатів. Ви побачите, що не достатньо просто отримати координати від моделі — треба коректно інтерпретувати їх, обрати формат та намалювати бокс з підписом, щоб мати можливість аналізувати якість детекцій та будувати власні кастомні інструменти перевірки та відлагодження моделей.

В першій частині ми розглянемо простий приклад отримання координат для обмежувальних рамок, а в другій частині як їх використати на зображенні.

2.1.1. Ручне визначення координат об’єкта

Для початку роботи із даним прикладом створіть новий файл, який буде містити код для ручного визначення об’єкта на зображенні.

Далі у новому файлі почніть введення коду із імпортування необхідної бібліотеки OpenCV та створення змінної, яка буде містити наші координати:

```
import cv2
```

```
points = []
```

Далі визначимо функцію – зворотний виклик, яка викликатися при натисненні на зображенні в певній точці:

```
def click_event(event, x, y, flags, params):
    global points
    if event == cv2.EVENT_LBUTTONDOWN:
        points.append((x,y))
        print(points)
```

Наступним кроком буде створення вікна для зображення, яке ми будемо аналізувати та вказати функцію зворотного виклику при кліку на зображенні:

```
img = cv2.imread("cat.jpg")
cv2.imshow("img", img)
cv2.setMouseCallback("img", click_event)
cv2.waitKey(0)
```

І останнім кроком буде додання коду для виведення координат, які можна застосовувати для другої частини прикладу:

```
(x1,y1),(x2,y2) = points
x,y = x1,y1
w,h = x2-x1, y2-y1
print("bbox:", x,y,w,h)
```

Виконайте даний код. З’явиться вікно із нашим зображенням, та курсором у вигляді координатної сітки. Клікніть у точці, яка на вашу думку відповідає за верхній лівий кут рамки, яка обводить об’єкт. Клікніть другий раз у точці, яка на вашу думку відповідає за нижній правий кут рамки, яка обводить об’єкт, та закрийте вікно

У консолі буде виведено рядок, який містить наші координати (x, y, w, h) – координати верхньої лівої точки, висота та ширина блоку:

```
bbox: 92 66 726 528
```

2.1.2. Ручне введення координат та візуалізація рамки

Другу частину даного прикладу розпочнемо зі створення нового файл, у який будемо додавати код для візуалізації об’єкта на зображенні.

Далі у новому файлі почніть введення коду із імпортування необхідної бібліотеки OpenCV:

```
import cv2
```

Далі вказуємо те саме зображення та координати, які ми отримали з попереднього кроку:

```
img = cv2.imread("cat.jpg")
boxes = [(92, 66, 726, 528)]  # (x,y,w,h)
labels = ["cat 0.99"]
```

Вказуємо також об’єкт, який ми розпізнали на зображенні, та показник нашої впевненості у точності нашого визначення об’єкта.

Далі додаємо код, який малює рамку навколо об’єкту, додає відповідний підпис та відкриває вікно:

```
for (x,y,w,h), lab in zip(boxes, labels):
    cv2.rectangle(img, (x,y), (x+w, y+h), (0,255,0), 2)
    cv2.putText(img, lab, (x, y-6), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
(0,0,255), 2)
```

```
cv2.imshow("Boxes", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Виконайте даний код. Проаналізуйте результат. Чи правильно ви вказали координати рамки?

2.2. Підрахунок на зображенні

У класичних методах комп'ютерного зору можна виконувати підрахунок об'єктів на зображенні без використання глибоких нейронних мереж. Один із базових підходів — це поєднання порогування та аналізу контурів: спочатку зображення переводять у чорно-білу маску за допомогою *thresholding*, а потім знаходять окремі об'єкти за допомогою функції *findContours*. Це дозволяє автоматично визначити кількість об'єктів, виділити їхні межі та візуалізувати їх на оригінальному зображенні. Такий метод добре підходить для простих, контрольованих сцен з однорідним фоном і є важливим прикладом того, як можна вирішувати задачі аналізу зображень без навчання моделі.

Для початку роботи із даним прикладом створіть новий файл, який буде містити код для підрахунку кількості об'єктів на зображенні.

Далі у новому файлі почніть введення коду із імпортування необхідної бібліотеки OpenCV:

```
import cv2
```

Далі вказуємо зображення, яке будемо використовувати для підрахунку об'єктів:

```
img = cv2.imread("9_examples/coffee.jpg")
```

Наступним етапом буде обробка зображення, а саме: переведення в grayscale, створення чорно-білої бінарної маски та знаходження контурів на зображенні:

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, 130, 255, cv2.THRESH_BINARY_INV)
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
```

Підраховуємо кількість об'єктів за контурами та виводимо у консолі:

```
count = len(contours)
print("Знайдено об'єктів:", count)
```

Для наочності візуалізовуємо знайдені контури на зображенні:

```
for i, cnt in enumerate(contours):
    x,y,w,h = cv2.boundingRect(cnt)
    cv2.rectangle(img, (x,y), (x+w,y+h), (0,255,0), 2)
    cv2.putText(img, str(i+1), (x,y-5), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
(0,255,0), 2)
cv2.imshow("Counted", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Виконайте даний код та проаналізуйте отримані результати. Чи були об'єкти правильно визначені (обведені) на зображенні? Виконайте даний код, змінивши вхідне зображення на будь-яке інше зображення із папки *9_examples* яке інше зображення та проаналізуйте результати.

2.3. Підрахунок на відео

Наступним прикладом буде теж підрахунок об’єктів, але вже у відео.

Для цього створіть новий файл та новий файл почніть введення коду із імпортування необхідної бібліотеки OpenCV:

```
import cv2
```

Далі ми вказуємо відео, яке будемо аналізувати:

```
cap = cv2.VideoCapture("planes.mp4")
```

Також, нам потрібно додати MOG2 фільтр. Це алгоритм сегментації фону/руху в відео, який використовується для виділення рухомих об’єктів. При цьому фон моделюється як комбінація (суміш) гаусівських розподілів, і з часом оновлюється. Пікселі, які не вписуються в модель фону, вважаються переднім планом.

```
fgbg = cv2.createBackgroundSubtractorMOG2(history=500, varThreshold=25,
detectShadows=False)
```

І вказуємо код, який відповідає за аналіз кадрів відео, попередньої підготовки кожного кадру, яка включає розмиття зображення (medianBlur), створення чорно-білої бінарної маски (threshold) та виділення контурів (findContours), як і у випадку із аналізом зображення:

```
while True:
    ok, frame = cap.read()
    if not ok: break
    fg = fgbg.apply(frame)
    fg = cv2.medianBlur(fg, 5)
    _, fg = cv2.threshold(fg, 200, 255, cv2.THRESH_BINARY)

    contours, _ = cv2.findContours(fg, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    count = 0
    for c in contours:
        x,y,w,h = cv2.boundingRect(c)
        if w*h < 500: continue
        count += 1
        cv2.rectangle(frame, (x,y), (x+w,y+h), (0,255,0), 2)

    cv2.putText(frame, f"Count: {count}", (10,30),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,255), 2)
    cv2.imshow("MOG2 count", frame)
    if cv2.waitKey(1) & 0xFF == 27: break

cap.release()
cv2.destroyAllWindows()
```

Виконайте даний код та проаналізуйте результати. Чи були об’єкти правильно визначені (обведені) на відео? Чи були шуми на відео, які були розпізнані як об’єкти?

2.4. Лінія перетину (line-crossing): простий трекінг центроїдів і підрахунок входів/виходів

У задачах аналізу відеопотоків дуже часто важливо не лише знаходити об’єкти, а й рахувати, скільки з них проходять через певну зону чи напрямок. Простий і наочний приклад — підрахунок людей, які входять і виходять із приміщення, або автомобілів, що проїжджають у певному напрямку. Для цього використовується концепція лінії

перетину: якщо центроїд об’єкта перетнув лінію зверху вниз — це, наприклад, “вхід”, якщо знизу вверх — “вихід”.

У даній підтемі ви навчитеся базовому трекінгу об’єктів через обчислення центроїдів (середніх координат box’a) та відстеженню їх зміни між кадрами. Це дозволяє мінімальними засобами — без складних deep-learning трекерів — зрозуміти фундаментальні принципи підрахунку рухомих об’єктів у потоці та побудувати просту систему аналітики з відеокамери.

Для цього створіть новий файл та новий файл почніть введення коду із імпортування необхідної бібліотеки OpenCV та бібліотеки для підрахунку дистанції між точками:

```
import cv2
from scipy.spatial.distance import cdist
```

Далі ми ініціалізує функцію assign_ids, яка:

- обчислює матрицю відстаней D між центроїдами попереднього кадру і поточного (cdist);
- жадібно (по зростанню дистанції) зводить пари (prev та curr), не перевищуючи max_dist;
- уже відомим об’єктам присвоює їх старі ID, а новим — видає нові;
- повертає список curr_ids у тому ж порядку, що й curr_centroids.

```
def assign_ids(prev_centroids, curr_centroids, prev_ids, max_dist=50):
    D = cdist(prev_centroids, curr_centroids) if len(prev_centroids)
and len(curr_centroids) else np.empty((0,0))
    assigned = {}
    used_curr = set()
    new_ids = {}
    next_id = (max(prev_ids)+1) if prev_ids else 0

    if D.size:
        for i in np.argsort(D, axis=None):
            r,c = divmod(i, D.shape[1])
            if D[r,c] > max_dist or r in assigned or c in used_curr:
continue
            assigned[r] = c; used_curr.add(c)

    for r, pid in enumerate(prev_ids):
        if r in assigned:
            new_ids[assigned[r]] = pid

    for c in range(len(curr_centroids)):
        if c not in new_ids:
            new_ids[c] = next_id; next_id += 1

    curr_ids = [new_ids[c] for c in range(len(curr_centroids))]
    return curr_ids
```

Далі ми проводимо читання відео та базову ініціалізацію змінних, де line_y - координата лінії підрахунку:

```
cap = cv2.VideoCapture("planes.mp4")
line_y = 300
```

```
prev_centroids, prev_ids = [], []
seen = {}
count_up, count_down = 0, 0
```

Наступним етапом є власне аналіз відеопотоку із відеофайлу. Сюди входить:

- попередня обробка кожного кадру, аналогічна обробці у попередньому прикладі;
- побудова рамок та центроїдів;
- присвоєння ID поточним об’єктам;
- перевірка перетину лінії та підрахунок напрямків;
- візуалізація рамок на відео;
- оновлення стану між кадрами:

```
while True:
    ok, frame = cap.read()
    if not ok:
        break
    if 'fgbg' not in globals():
        fgbg = cv2.createBackgroundSubtractorMOG2(history=300,
varThreshold=25, detectShadows=False)
    fg = fgbg.apply(frame)
    fg = cv2.medianBlur(fg, 5)
    _, fg = cv2.threshold(fg, 200, 255, cv2.THRESH_BINARY)
    contours,_ = cv2.findContours(fg, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

    curr_boxes, curr_centroids = [], []
    for c in contours:
        x,y,w,h = cv2.boundingRect(c)
        if w*h < 800: continue
        curr_boxes.append((x,y,w,h))
        curr_centroids.append((x+w/2, y+h/2))

    # зіставлення ID
    curr_ids = assign_ids(prev_centroids, curr_centroids, prev_ids,
max_dist=60)

    # перевірка перетину лінії
    for (x,y,w,h), (cx,cy), oid in zip(curr_boxes, curr_centroids,
curr_ids):
        prev_y = seen.get(oid, cy)
        if prev_y <= line_y < cy:   # вниз
            count_down += 1
        elif prev_y >= line_y > cy: # вгору
            count_up += 1
        seen[oid] = cy
        cv2.rectangle(frame,(x,y),(x+w,y+h),(0,255,0),2)
        cv2.putText(frame, f"ID {oid}", (x, y-6),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,0,255),2)
        cv2.circle(frame, (int(cx),int(cy)), 3, (255,0,0), -1)

    # лінія та лічильники
```

```
            cv2.line(frame, (0,line_y), (frame.shape[1], line_y), (0,255,255),
2)
            cv2.putText(frame, f"UP: {count_up}  DOWN: {count_down}", (10,30),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (255,255,0),2)

            cv2.imshow("Line counting", frame)
            if cv2.waitKey(1) & 0xFF == 27: break

            prev_centroids = curr_centroids
            prev_ids = curr_ids

        cap.release()
        cv2.destroyAllWindows()
```

Виконайте даний код та проаналізуйте результати. Наскільки точно спрацьовує алгоритм та чи точно веде підрахунок об’єктів, які перетинають лінію. Які можуть бути причини неправильного підрахунку? Спробуйте виконати код з іншим відео, та змінити координати лінії та проаналізуйте результати.